Relational Algebra

CSE 341
Introduction to Database Systems
Winter 2024
© Royal Military College of Canada

Introduction

- Relational calculus and relational algebra are two foundational concepts in database theory, essential for understanding how queries are formulated and processed in a relational database system.

- Both concepts are important in understanding the theoretical foundation of SQL and other database query languages, providing a framework for understanding how queries are logically constructed and executed.

Introduction

- Relational algebra and relational calculus are formal languages associated with the relational model.

- Informally, relational algebra is a (high-level) procedural language and relational calculus a non-procedural language.

- However, formally both are equivalent to one another.

- A language that produces a relation that can be derived using relational calculus is <u>relationally complete</u>.

Relational Algebra

- The relational algebra is a theoretical language with operations that work on one or more relations to define another relation without changing the original relation(s).

- Both the operands and the results are relations.

- The output from one operation can become the input to another operation.

- This ability allows expressions to be nested in the relational algebra, just as we can nest arithmetic operations. This property is called closure

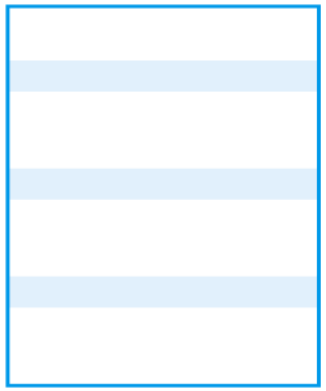
Relational Algebra Operations

- Five basic operations in relational algebra: **Selection**, **Projection**, **Cartesian product**, **Union**, and **Set Difference**.

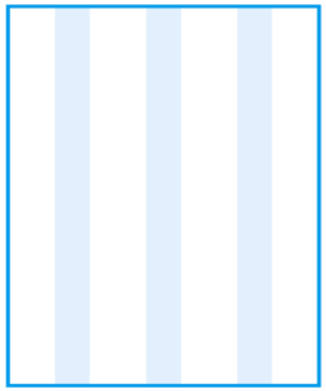
- These perform most of the data retrieval operations needed.

- Also have **Join**, **Intersection**, and **Division** operations, which can be expressed in terms of 5 basic operations.

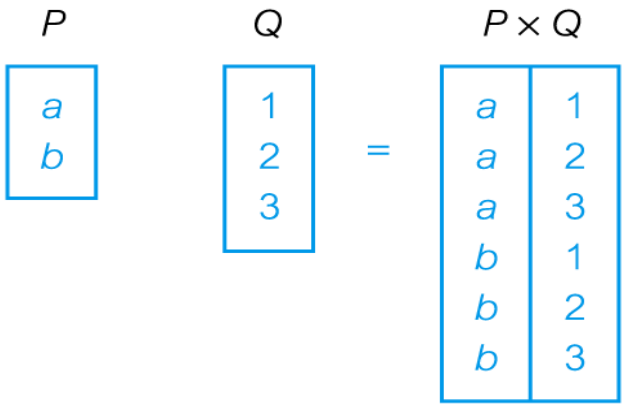
Relational Algebra Operations



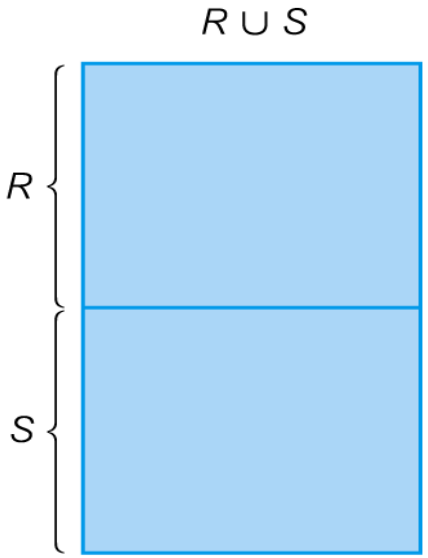
(a) Selection



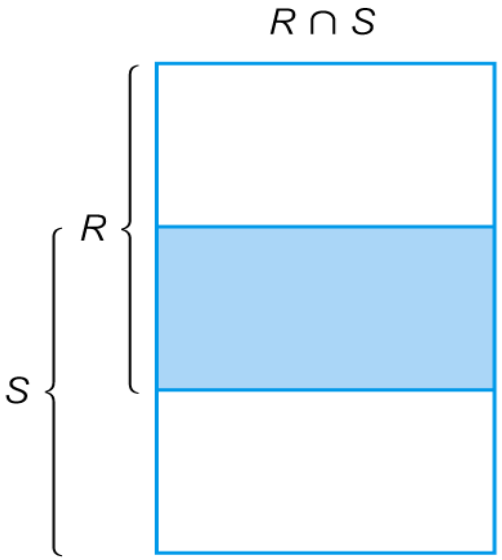
(b) Projection



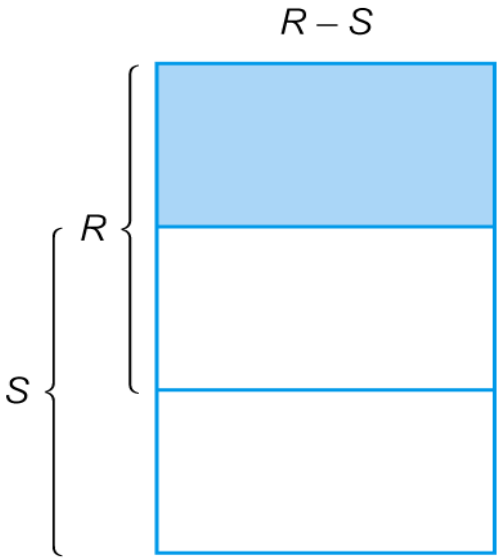
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

Relational Algebra Operations

T

A	B
a	1
b	2

U

B	C
1	x
1	y
3	z

$T \bowtie U$

A	B	C
a	1	x
a	1	y

(g) Natural join

$T \rhd_B U$

A	B
a	1

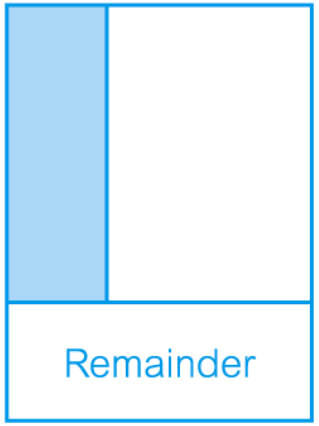
(h) Semijoin

$T \rtimes_C U$

A	B	C
a	1	x
a	1	y
b	2	

(i) Left Outer join

R



S

$R \div S$

(j) Divis on (shaded area)

V

A	B
a	1
a	2
b	1
b	2
c	1

W

B
1
2

$V \div W$

A
a
b

Example of division

Selection (or Restriction)

- $\sigma_{predicate}$ (R)
  - Works on a single relation R and defines a relation that contains only those tuples (rows) of R that satisfy the specified condition (*predicate*).

Example - Selection (or Restriction)

- List all staff with a salary greater than £10,000.

$\sigma_{salary > 10000}$ **(Staff)**

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24- Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Projection

- $\Pi_{col_1, \ldots, col_n}(R)$
  - Works on a single relation R and defines a relation that contains a vertical subset of R, extracting the values of specified attributes and eliminating duplicates.

Example - Projection

- Produce a list of salaries for all staff, showing only staffNo, fName, lName, and salary details.

$$\Pi_{\textbf{staffNo, fName, lName, salary}}(\textbf{Staff})$$

staffNo	fName	lName	salary
SL21	John	White	30000
SG37	Ann	Beech	12000
SG14	David	Ford	18000
SA9	Mary	Howe	9000
SG5	Susan	Brand	24000
SL41	Julie	Lee	9000

Union

- $R \cup S$
  - Union of two relations R and S defines a relation that contains all the tuples of R, or S, or both R and S, duplicate tuples being eliminated.
  - R and S must be union-compatible.

- If R and S have *I* and *J* tuples, respectively, union is obtained by concatenating them into one relation with a maximum of $(I + J)$ tuples.

Example - Union

- List all cities where there is either a branch office or a property for rent.

$$\Pi_{\textbf{city}}(\textbf{Branch}) \cup \Pi_{\textbf{city}}(\textbf{PropertyForRent})$$

city
London
Aberdeen
Glasgow
Bristol

Set Difference

- R – S
  - Defines a relation consisting of the tuples that are in relation R, but not in S.
  - R and S must be union-compatible.

Example - Set Difference

- List all cities where there is a branch office but no properties for rent.

$$\Pi_{\text{city}}(\textbf{Branch}) - \Pi_{\text{city}}(\textbf{PropertyForRent})$$

city
Bristol

Intersection

- $R \cap S$
  - Defines a relation consisting of the set of all tuples that are in both R and S.
  - R and S must be union-compatible.

- Expressed using basic operations:

  $R \cap S = R - (R - S)$

Example - Intersection

- List all cities where there is both a branch office and at least one property for rent.

$$\Pi_{city}(\text{Branch}) \cap \Pi_{city}(\text{PropertyForRent})$$

city
Aberdeen
London
Glasgow

Cartesian product

- R X S
  - Defines a relation that is the concatenation of every tuple of relation R with every tuple of relation S.

Example - Cartesian product

- List the names and comments of all clients who have viewed a property for rent.

$$(\Pi_{\text{clientNo, fName, lName}}(\textbf{Client})) \times (\Pi_{\text{clientNo, propertyNo, comment}}(\textbf{Viewing}))$$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR56	PA14	too small
CR76	John	Kay	CR76	PG4	too remote
CR76	John	Kay	CR56	PG4	
CR76	John	Kay	CR62	PA14	no dining room
CR76	John	Kay	CR56	PG36	
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR62	PA14	no dining room
CR56	Aline	Stewart	CR56	PG36	
CR74	Mike	Ritchie	CR56	PA14	too small
CR74	Mike	Ritchie	CR76	PG4	too remote
CR74	Mike	Ritchie	CR56	PG4	
CR74	Mike	Ritchie	CR62	PA14	no dining room
CR74	Mike	Ritchie	CR56	PG36	
CR62	Mary	Tregear	CR56	PA14	too small
CR62	Mary	Tregear	CR76	PG4	too remote
CR62	Mary	Tregear	CR56	PG4	
CR62	Mary	Tregear	CR62	PA14	no dining room
CR62	Mary	Tregear	CR56	PG36	

Example - Cartesian product and Selection

- Use selection operation to extract those tuples where Client.clientNo = Viewing.clientNo.

$$\sigma_{\text{Client.clientNo = Viewing.clientNo}}((\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \times (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing})))$$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

Cartesian product and Selection can be reduced to a single operation called a *Join*.

Join Operations

- Join is a derivative of Cartesian product.
- Equivalent to performing a Selection, using join predicate as selection formula, over Cartesian product of the two operand relations.
- One of the most difficult operations to implement efficiently in an RDBMS and one reason why RDBMSs have intrinsic performance problems.

Join Operations

- Various forms of join operation
  - Theta join
  - Equijoin (a particular type of Theta join)
  - Natural join
  - Outer join
  - Semijoin

Theta join (θ-join)

- $R \bowtie_F S$
  - Defines a relation that contains tuples satisfying the predicate F from the Cartesian product of R and S.
  - The predicate F is of the form $R.a_i \ \theta \ S.b_i$ where θ may be one of the comparison operators ($<, \leq, >, \geq, =, \neq$).

Theta join (θ-join)

- Can rewrite Theta join using basic Selection and Cartesian product operations.

$$\mathbf{R} \bowtie_{\mathbf{F}} \mathbf{S} = \sigma_{\mathbf{F}}(\mathbf{R} \times \mathbf{S})$$

Degree of a Theta join is sum of degrees of the operand relations R and S. If predicate F contains only equality (=), the term *Equijoin* is used.

Example - Equijoin

- List the names and comments of all clients who have viewed a property for rent.

$$(\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \bowtie_{\text{Client.clientNo = Viewing.clientNo}} (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

Natural join

- **R ⋈ S**
  - An Equijoin of the two relations R and S over all common attributes *x*. One occurrence of each common attribute is eliminated from the result.

Example - Natural join

- List the names and comments of all clients who have viewed a property for rent.

$$(\Pi_{\text{clientNo, fName, lName}}(\textbf{Client})) \bowtie (\Pi_{\text{clientNo, propertyNo, comment}}(\textbf{Viewing}))$$

clientNo	fName	lName	propertyNo	comment
CR76	John	Kay	PG4	too remote
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR56	Aline	Stewart	PG36	
CR62	Mary	Tregear	PA14	no dining room

Outer join

- To display rows in the result that do not have matching values in the join column, use Outer join.

- **R ⟕ S**
  - (Left) outer join is join in which tuples from R that do not have matching values in common columns of S are also included in result relation.

Example - Left Outer join

- Produce a status report on property viewings.

$$\Pi_{\text{propertyNo, street, city}}(\textbf{PropertyForRent}) \mathbin{⟕} \textbf{Viewing}$$

propertyNo	street	city	clientNo	viewDate	comment
PA14	16 Holhead	Aberdeen	CR56	24-May-01	too small
PA14	16 Holhead	Aberdeen	CR62	14-May-01	no dining room
PL94	6 Argyll St	London	**null**	**null**	**null**
PG4	6 Lawrence St	Glasgow	CR76	20-Apr-01	too remote
PG4	6 Lawrence St	Glasgow	CR56	26-May-01	
PG36	2 Manor Rd	Glasgow	CR56	28-Apr-01	
PG21	18 Dale Rd	Glasgow	**null**	**null**	**null**
PG16	5 Novar Dr	Glasgow	**null**	**null**	**null**

Semijoin

- $\mathbf{R} \rhd_{\mathbf{F}} \mathbf{S}$
  - Defines a relation that contains the tuples of R that participate in the join of R with S.

Can rewrite Semijoin using Projection and Join:

$$\mathbf{R} \rhd_{\mathbf{F}} \mathbf{S} = \Pi_{\mathbf{A}}(\mathbf{R} \bowtie_{\mathbf{F}} \mathbf{S})$$

Example - Semijoin

- **List complete details of all staff who work at the branch in Glasgow.**

$$\textbf{Staff} \ltimes_{\textbf{Staff.branchNo=Branch.branchNo}} (\sigma_{\textbf{city='Glasgow'}}(\textbf{Branch}))$$

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24- Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Division

- **R ÷ S**
  - Defines a relation over the attributes C that consists of set of tuples from R that match combination of *every* tuple in S.

- Expressed using basic operations:

$$\mathbf{T_1 \leftarrow \Pi_C(R)}$$

$$\mathbf{T_2 \leftarrow \Pi_C((S \times T_1) - R)}$$

$$\mathbf{T \leftarrow T_1 - T_2}$$

Example - Division

- Identify all clients who have viewed all properties with three rooms.

$$(\Pi_{\text{clientNo, propertyNo}}(\textbf{Viewing})) \div (\Pi_{\text{propertyNo}}(\sigma_{\text{rooms = 3}}\ (\textbf{PropertyForRent})))$$

$\Pi_{\text{clientNo,propertyNo}}(\text{Viewing})$

clientNo	propertyNo
CR56	PA14
CR76	PG4
CR56	PG4
CR62	PA14
CR56	PG36

$\Pi_{\text{propertyNo}}(\sigma_{\text{rooms=3}}(\text{PropertyForRent}))$

propertyNo
PG4
PG36

RESULT

clientNo
CR56

Aggregate Operations

- $_{AL}(R)$
  - Applies aggregate function list, AL, to R to define a relation over the aggregate list.
  - AL contains one or more (<aggregate_function>, <attribute>) pairs .
- Main aggregate functions are: COUNT, SUM, AVG, MIN, and MAX.

Example – Aggregate Operations

- How many properties cost more than £350 per month to rent?

$$\rho_{R}(\textbf{myCount})\ \textbf{COUNT propertyNo}\ (\sigma_{\textbf{rent > 350}}\ (\textbf{PropertyForRent}))$$

myCount
5

(a)

Grouping Operation

- $_{GA}\ _{AL}(R)$
  - Groups tuples of R by grouping attributes, GA, and then applies aggregate function list, AL, to define a new relation.
  - AL contains one or more (<aggregate_function>, <attribute>) pairs.
  - Resulting relation contains the grouping attributes, GA, along with results of each of the aggregate functions.

Example – Grouping Operation

- **Find the number of staff working in each branch and the sum of their salaries.**

$$\rho_R(\textbf{branchNo, myCount, mySum})$$

$$_{\textbf{branchNo}}\ \textbf{COUNT staffNo, SUM salary}\ (\textbf{Staff})$$

branchNo	myCount	mySum
B003	3	54000
B005	2	39000
B007	1	9000

Next…

Relational Calculus